

California State University Long Beach
Computer Engineering and Computer Science Department
CECS 440 – Computer Architecture
Lab 10 – MIPS Execute Stage

Objectives:

- Implement the **Execute (EX)** stage of a single-cycle MIPS processor in Verilog (ALU, branch adder, ALUSrc and RegDst multiplexers).
- Use **ALUControlE** to determine the ALU operation for R-type, BEQ, and load/store instructions.
- Manage operand selection using **ALUSrcE** (register vs. immediate) and **RegDstE** (RT vs. RD) to produce the correct ALU input and destination register.
- Compute the **branch target address** and **ZeroE** flag for BEQ instruction evaluation.
- Pass control signals (**RegWriteE**, **MemtoRegE**, **MemWriteE**, **BranchE**) to the next stage without modification.
- Simulate and verify correct ALU results, branch address computation, and signal propagation using **Xilinx Vivado** behavioral simulation with a Verilog testbench.

Background:

In the single cycle MIPS processor, there are 5 stages of execution. The Execute stage primarily involves the ALU which is used to calculate the result of R-Type instructions, performs subtraction to check equality of operands of BEQ, and calculates address for load/store operations.

The signals involved in this stage are:

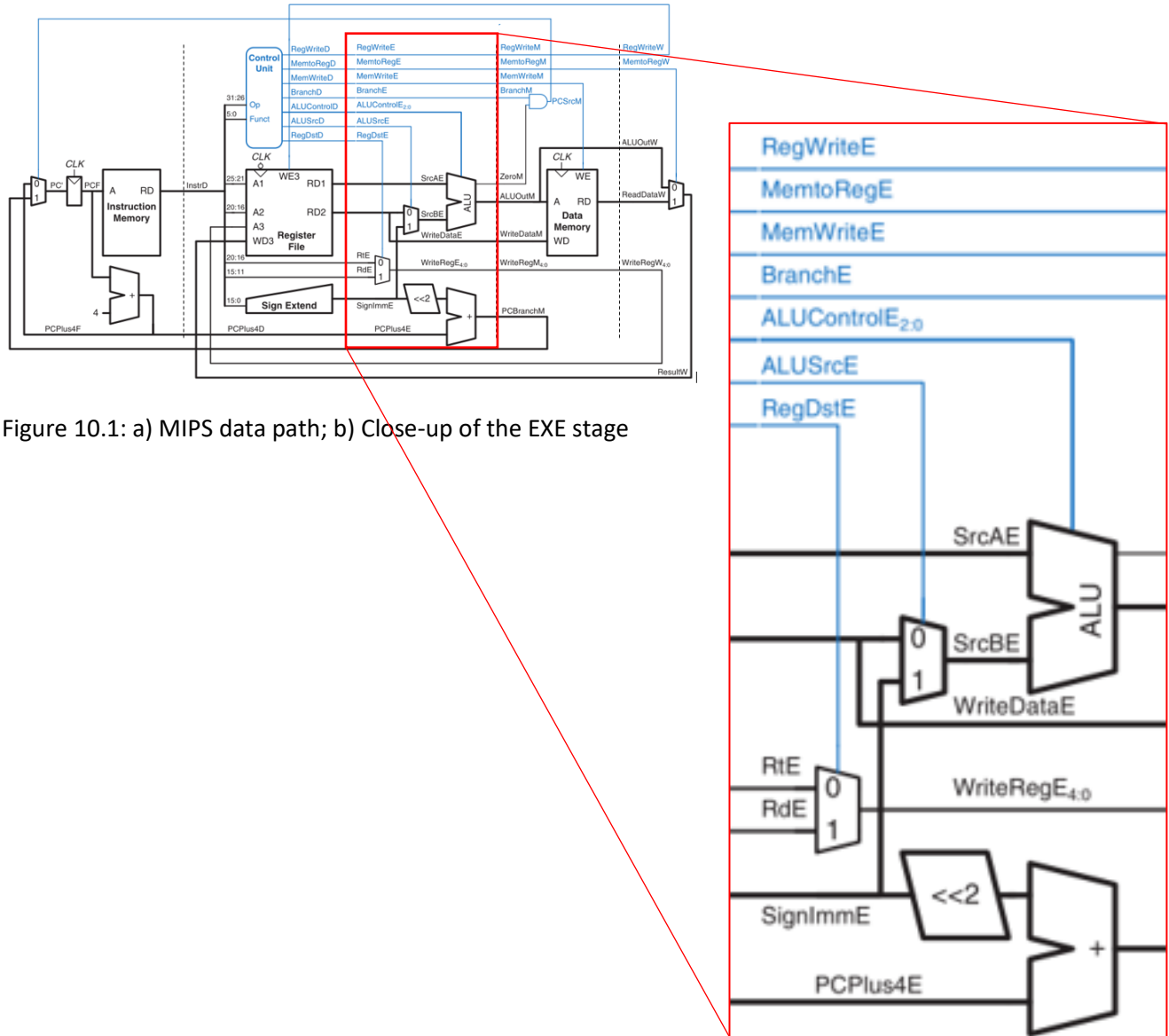
RegWriteE, MemtoRegE, MemWriteE, and BranchE signals are passed along but do not control any hardware in the EX stage.

ALUControlE: makes the ALU perform a specific operation

ALUSrcE: when equal to 0 the ALU input B receives output of register file; when equal to 1 the ALU input B receives immediate data

RegDstE: When equal to 0 then **rt** will choose the register to be written for the current instruction's writeback stage; when equal to 1 then **rd** will choose the destination register (for R-Type instructions)

The adder (in the bottom of Figure 10.1) is used to calculate the branch target address (only used in the beq instruction).



Steps:

1. Get the mux2.v file from a previous lab.
2. Get the 32-bit adder from a previous lab.
3. A left shifter is needed. Use **sl2.v** with Verilog module definition given below:

```
1 module sl2(a, y);  
2     input  [31:0] a;  
3     output [31:0] y;  
4     assign y = {    };  
5 endmodule
```

Figure 10.2: Shift-left module

4. Create Arithmetic Logic Unit in a file named **alu.v** and name the module **alu**:

```
1 module alu(srca, srcb, alucontrol, aluout, zero);  
2     input [31:0] srca, srcb;  
3     input [2:0] alucontrol;  
4     output [31:0] aluout;  
5     output zero;  
6  
7     assign aluout = (alucontrol == 0) ? :  
8                     (alucontrol == 1) ? :  
9                     (alucontrol == 2) ? :  
10                    (alucontrol == 6) ? :  
11                    (alucontrol == 7) ? :  
12                    32'hx; // default aluout value  
13  
14     assign zero = ;  
15 endmodule
```

Figure 10.3: ALU module

5. Use the block diagram on page 1 and complete the module skeleton in Figure 10.4 to create the IF stage:

```

6 module EX_Stage(
7     regwrited,
8     memtoregd,
9     memwrited,
10    branchd,
11    alucontrole,
12    alusrce,
13    regdste,
14    srcae,
15    writedatad,
16    rte,
17    rde,
18    signimme,
19    pcplus4e,
20    regwritee,
21    memtorege,
22    memwritee,
23    branche,
24    zeroe,
25    aluoute,
26    writedatae,
27    writerege,
28    pcbranche
29 );
30 input  regwrited, memtoregd, memwrited,
31        branchd, alusrce, regdste;
32 input  [2:0]    alucontrole;
33 input  [31:0]   srcae, writedatad, signimme, pcplus4e;
34 input  [4:0]    rte, rde;
35 output regwritee, memtorege, memwritee, branche, zeroe;
36 output [31:0]   aluoute, writedatae, pcbranche;
37 output [4:0]    writerege;
38
39 wire   [31:0]   srcbe, signimmshe;
40
41 assign regwritee = regwrited,
42        memtorege = memtoregd,
43        memwritee = memwrited,
44        branche   = branchd,
45        writedatae = writedatad;
46
47 alu alu( );
48
49 mux2 #(32) srcbmux( );
50
51 mux2 #(5) wrmux( );
52
53 s12 immsh( );
54
55 adder32 pcadd2( );
56 endmodule

```

Figure 10.4: Skeleton of the EXE_Stage module

- Test your design to ensure it works correctly using the Verilog test fixture shown in the Figure 10.5. The expected output is shown in Figure 10.6.

```

1 module t_EX_Stage;
2     reg regwrited, memtoregd, memwrited, branchd, alusrce, regdste;
3     reg [2:0] alucontrole;
4     reg [31:0] srcae, writedatad, signimme, pcplus4e;
5     reg [4:0] rte, rde;
6     wire regwritee, memtorege, memwritee, branche, zeroe;
7     wire [31:0] aluoute, writedatae, pcbranche;
8     wire [4:0] writerege;
9     integer i;
10
11     EX_Stage dut(
12         regwrited, memtoregd, memwrited, branchd,
13         alucontrole, alusrce, regdste, srcae,
14         writedatad, rte, rde, signimme, pcplus4e,
15         regwritee, memtorege, memwritee, branche,
16         zeroe, aluoute, writedatae, writerege,
17         pcbranche
18     );
19
20     initial begin
21         srcae = 32'h00001212;
22         writedatad = 32'h00003434;
23         signimme = 32'hffffffff;
24         pcplus4e = 32'h44444444;
25
26         for (i = 0; i < 8; i = i + 1)
27             testcase;
28         #10 $finish;
29     end
30
31     task testcase;
32     begin
33         {regwrited, memtoregd, memwrited, branchd, alusrce, regdste} = $random;
34         alucontrole = i;
35         case (alucontrole)
36             0: $display("ALU is performing AND");
37             1: $display("ALU is performing OR");
38             2: $display("ALU is performing ADD");
39             6: $display("ALU is performing SUB");
40             7: $display("ALU is performing SLT");
41             default: $display("ALU Operation is invalid");
42         endcase
43
44         #1 $display("aluoute = %h", aluoute);
45         if (zeroe != (!aluoute))
46             $display("zero flag is malfunctioning");
47
48         if (regwritee != regwrited || memtorege != memtoregd ||
49             memwritee != memwrited || branche != branchd ||
50             writedatae != writedatad) begin
51             $display("Control signals did not pass correctly");
52             $finish;
53         end
54
55         if (pcbranche != ({signimme[29:0], 2'b00} + pcplus4e)) begin
56             $display("Branch Adder is malfunctioning");
57             $finish;
58         end
59
60         if (i == 3)
61             i = 5;
62     end
63 endtask
64 endmodule

```

Figure 10.5: Testbench for the EXE_Stage module

```
ALU is performing AND
aluoute = 00001010
ALU is performing OR
aluoute = 00003636
ALU is performing ADD
aluoute = 00004646
ALU Operation is invalid
aluoute = xxxxxxxx
ALU is performing SUB
aluoute = ffffddde
ALU is performing SLT
aluoute = 00000001
```

Done

Figure 10.6: Expected output for the Lab 7

WHAT TO SUBMIT

- Txt File: paste **ALL** .v files into a plain text file named **lab6-sources.txt**.
 - Submit aludec.v, maindec.v, design.v, design_tb.v
 - **NO ZIP FILES PLEASE! You will receive 0 credit for source files.**
- Lab report using your **course template**, including:
 - brief design notes and any control tables used,
 - screenshots of key Vivado waveforms or console output validating correctness
 - Include details in all aspects (Steps, Results, etc.) of the template for full credit!
 - What is the role of the Execute (EX) Stage in a MIPS Processor for each instruction type?
 - What is the function of control signals in the EX-stage for each instruction type?

Upload to Canvas. Keep your files for future labs.

NOTE: keep these lab files as they will be needed for future labs!

